NoSQL Database

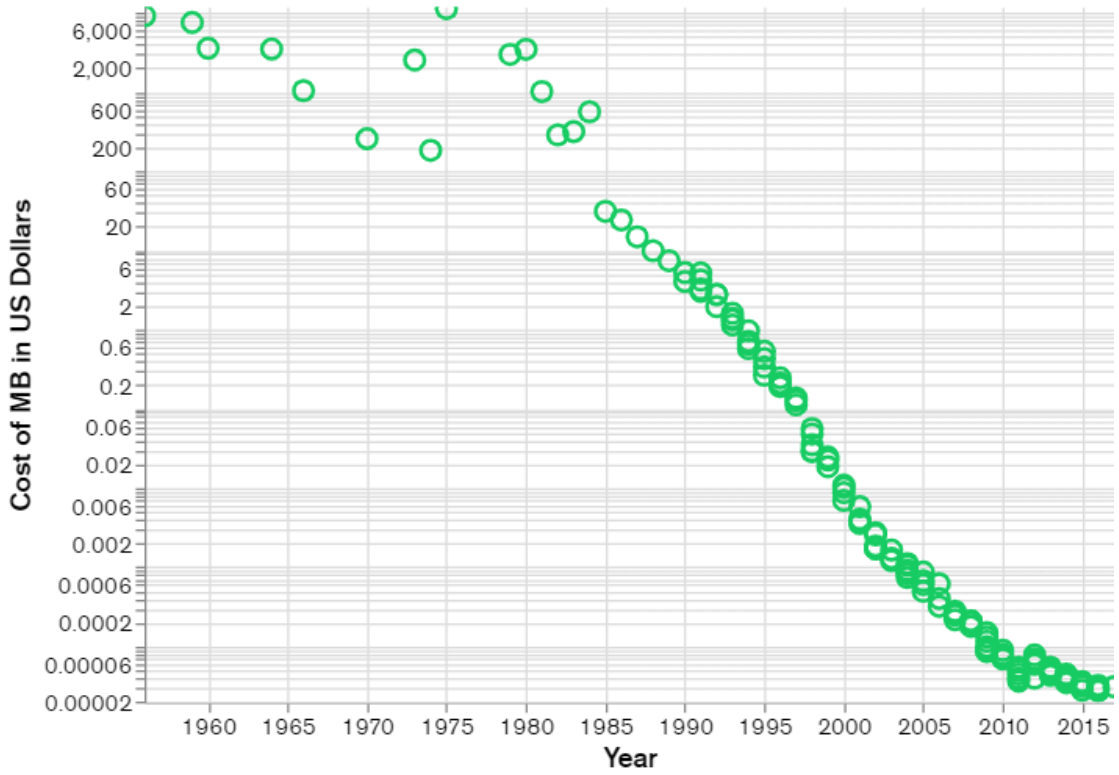
Database II - Lecture 7

1. What is a NoSQL database?

When people use the term "NoSQL database", they typically use it to refer to any non-relational database. Some say the term "NoSQL" stands for "non SQL" while others say it stands for "not only SQL". Either way, most agree that NoSQL databases are databases that store data in a format other than relational tables. NoSQL, as opposed to SQL, is a database management approach, whereas SQL is just a query language, similar to the query languages of NoSQL databases.

2. Brief history of NoSQL databases

NoSQL databases emerged in the late 2000s as the cost of storage dramatically decreased. Gone were the days of needing to create a complex, difficult-to-manage data model in order to avoid data duplication. Developers (rather than storage) were becoming the primary cost of software development, so NoSQL databases optimized for developer productivity.



As storage costs rapidly decreased, the amount of data that applications needed to store and query increased. NoSQL databases allow developers to store huge amounts of unstructured data, giving them a lot of flexibility.

Cloud computing also rose in popularity, and developers began using public clouds to host their applications and data. They wanted the ability to distribute data across multiple servers and regions

to make their applications resilient, to scale out instead of scale up, and to intelligently geo-place their data. Some NoSQL databases like MongoDB provide these capabilities.

3. NoSQL database features

Each NoSQL database has its own unique features. At a high level, many NoSQL databases have the following features:

- Flexible schemas
- Horizontal scaling
- Fast queries due to the data model
- Ease of use for developers

4. Types of NoSQL databases

Over time, four major types of NoSQL databases emerged: document databases, key-value databases, wide-column stores, and graph databases.

4.1 Document-oriented databases

A document-oriented database stores data in documents similar to JSON (JavaScript Object Notation) objects. Each document contains pairs of fields and values. The values can typically be a variety of types, including things like strings, numbers, booleans, arrays, or even other objects. A document database offers a flexible data model, much suited for semi-structured and typically unstructured data sets. They also support nested structures, making it easy to represent complex relationships or hierarchical data.

Examples of document databases are MongoDB and Couchbase. A typical document will look like the following:

```
{
  "_id": "12345",
  "name": "foo bar",
  "email": "foo@bar.com",
  "address": {
    "street": "123 foo street",
    "city": "some city",
    "state": "some state",
    "zip": "123456"
  },
  "hobbies": ["music", "guitar", "reading"]
}
```

4.2 Key-value databases

A key-value store is a simpler type of database where each item contains keys and values. Each key is unique and associated with a single value. They are used for caching and session management and provide high performance in reads and writes because they tend to store things in memory. Examples are Amazon DynamoDB and Redis. A simple view of data stored in a key-value database is given below:

```
Key: user:12345

Value: {"name": "foo bar", "email": "foo@bar.com",
"designation": "software developer"}
```

4.3 Wide-column stores

Wide-column stores store data in tables, rows, and dynamic columns. The data is stored in tables. However, unlike traditional SQL databases, wide-column stores are flexible, where different rows can have different sets of columns. These databases can employ column compression techniques to reduce the storage space and enhance performance. The wide rows and columns enable efficient retrieval of sparse and wide data. Some examples of wide-column stores are Apache Cassandra and HBase. A typical example of how data is stored in a wide-column is as follows:

name	id	email	dob	city
Foo bar	12345	foo@bar.com		Some city
Carn Yale	34521	bar@foo.com	12-05-1972	

4.4 Graph databases

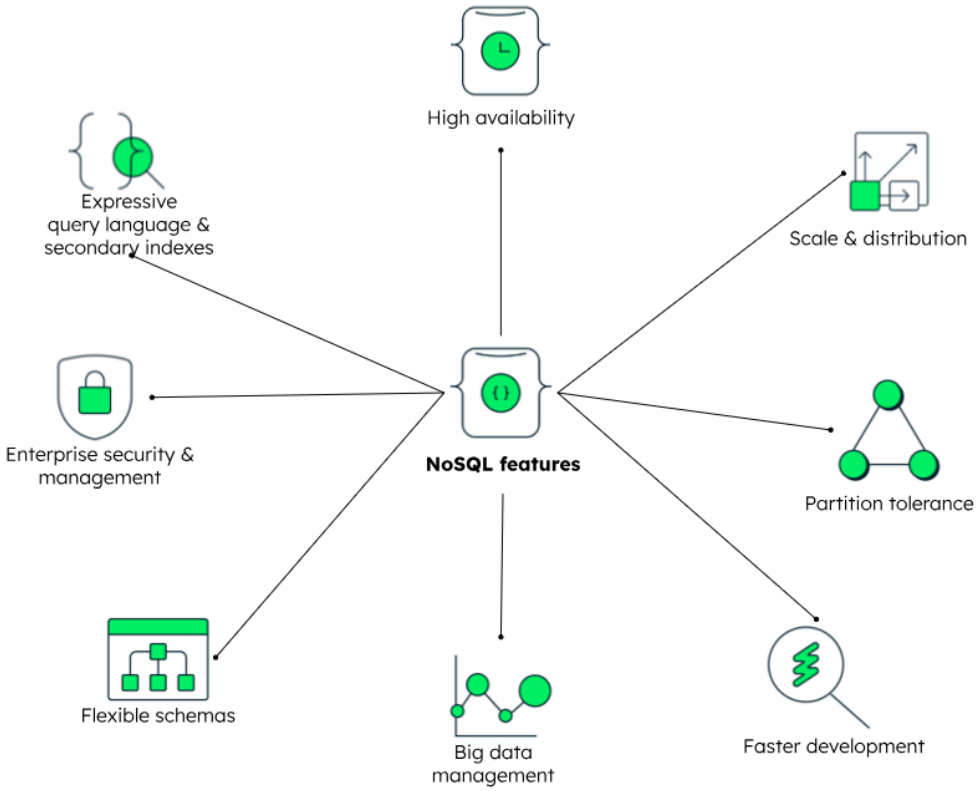
A graph database stores data in the form of nodes and edges. Nodes typically store information about people, places, and things (like nouns), while edges store information about the relationships between the nodes. They work well for highly connected data, where the relationships or patterns may not be very obvious initially.

4.5 Multi-model databases

Multi-model databases support more than one type of NoSQL data model so that developers can choose based on their application requirements. These databases have a unified database engine that can handle multiple data models within a database instance. Examples are CosmosDB and ArangoDB.

5. NoSQL database features

NoSQL databases are flexible, scalable, and distributed databases. Different types of NoSQL databases have their own unique features.



6. BASE compliance

NoSQL databases are BASE compliant, i.e., basic availability soft state eventual consistency. Basic availability refers to the ability of the system to tolerate a partial failure (like a loss of a node). Soft state means that the system allows temporary inconsistencies before eventually achieving consistency automatically over time. BASE compliance ensures high availability, faster data processing, scalability, and flexibility. However, MongoDB can also be configured to provide multi-document ACID compliance.

6. Difference between RDBMS and NoSQL databases

While a variety of differences exist between relational database management systems (RDBMS) and NoSQL databases, one of the key differences is the way the data is modeled in the database. In this section, we'll work through an example of modeling the same data in a relational database and a NoSQL database. Then, we'll highlight some of the other key differences between relational databases and NoSQL databases.

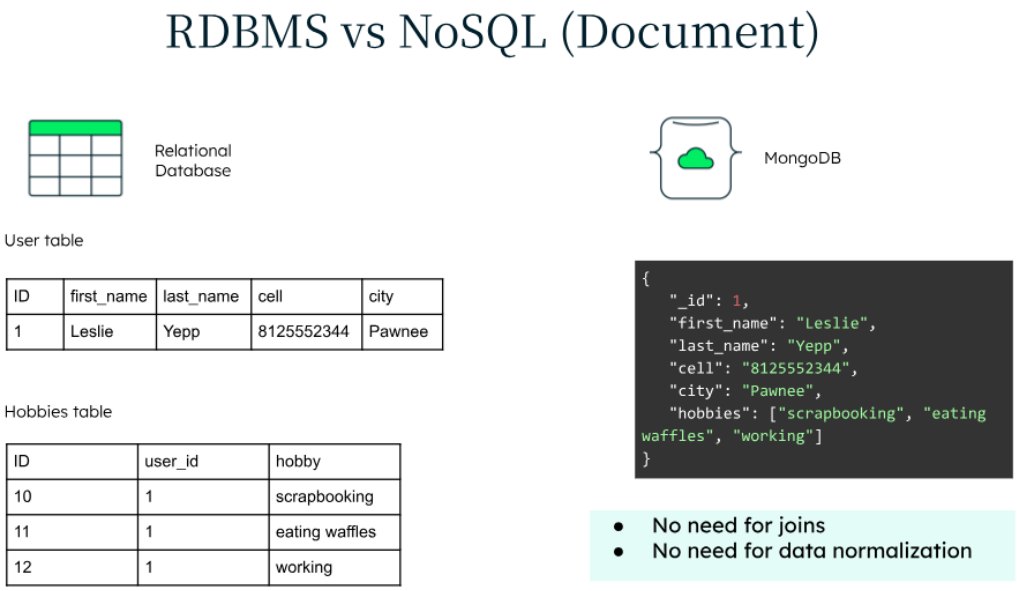
6.1 RDBMS vs NoSQL: Data Modeling Example

Let's consider an example of storing information about a user and their hobbies. We need to store a user's first name, last name, cell phone number, city, and hobbies.

In a relational database management system (RDBMS), we'd likely create two tables: one for Users and one for Hobbies. In order to retrieve all of the information about a user and their hobbies, information from the Users table and Hobbies table will need to be joined together.

The data model we design for a NoSQL database will depend on the type of NoSQL database we choose. Let's consider how to store the same information about a user and their hobbies in a document database like MongoDB.

In order to retrieve all of the information about a user and their hobbies, a single document can be retrieved from the database. No joins are required, resulting in faster queries.



When should NoSQL be used?

When deciding which database to use, decision-makers typically find one or more of the following factors that lead them to select a NoSQL database:

- Fast-paced Agile development
- Storage of structured and semi-structured data
- Huge volumes of data
- Requirements for scale-out architecture
- Modern application paradigms like microservices and real-time streaming

6.2 Differences between RDBMS and NoSQL databases

There are a variety of differences between relational database management systems and non-relational databases. One of the key differences is the way data is modeled in the database. Some key differences of each feature is listed below:

a) Data modeling

NoSQL: Data models vary based on the type of NoSQL database used — for example, key-value, document, graph, and wide-column — making the model suitable for semi-structured and unstructured data.

RDBMS: RDBMS uses a tabular data structure, with data represented as a set of rows and columns, making the model suitable for structured data.

b) Schema

NoSQL: It provides a flexible schema where each set of documents/row-column/key-value pairs can contain different types of data. It's easier to change schema, if required, due to the flexibility.

RDBMS: This is a fixed schema where every row should contain the same predefined column types. It is difficult to change the schema once data is stored.

c) Query language

NoSQL: It varies based on the type of NoSQL database used. For example, MongoDB has MQL, and Neo4J uses Cypher.

RDBMS: This uses structured query language (SQL).

d) Scalability

NoSQL: NoSQL is designed for vertical and horizontal scaling.

RDBMS: RDBMS is designed for vertical scaling. However, it can extend limited capabilities for horizontal scaling.

e) Data relationships

NoSQL: Relationships can be nested, explicit, or implicit.

RDBMS: Relationships are defined through foreign keys and accessed using joins.

f) Transaction type

NoSQL: Transactions are either ACID- or BASE-compliant.

RDBMS: Transactions are ACID-compliant.

g) Performance

NoSQL: NoSQL is suitable for real-time processing, big data analytics, and distributed environments.

RDBMS: RDBMS is suitable for read-heavy and transaction workloads.

h) Data consistency

NoSQL: This offers eventual consistency, in most cases.

RDBMS: This offers high data consistency.

i) Distributed computing

NoSQL: One of the main reasons to introduce NoSQL was for distributed computing, and NoSQL databases support distributed data storage, vertical and horizontal scaling through sharding, replication, and clustering.

RDBMS: RDBMS supports distributed computing through clustering and replication. However, it's less scalable and flexible as it's not traditionally designed to support distributed architecture.

j) Fault tolerance

NoSQL: NoSQL has built-in fault tolerance and high availability due to data replication.

RDBMS: RDBMS uses replication, backup, and recovery mechanisms. However, as they are designed for these, additional measures like disaster recovery mechanisms may need to be implemented during application development.

k) Data partitioning

NoSQL: It's done through sharding and replication.

RDBMS: It supports table-based partitioning and partition pruning.

l) Data to object mapping

NoSQL: NoSQL stores the data in a variety of ways — for example, as JSON documents, wide-column stores, or key-value pairs. It provides abstraction through the ODM (object-data mapping) frameworks to work with NoSQL data in an object-oriented manner.

RDBMS: RDBMS relies more on data-to-object mapping so that there is seamless integration between the database columns and the object-oriented application code.